Desenvolvimento Web 1

Prof. Diego Max

Aula 03.1:

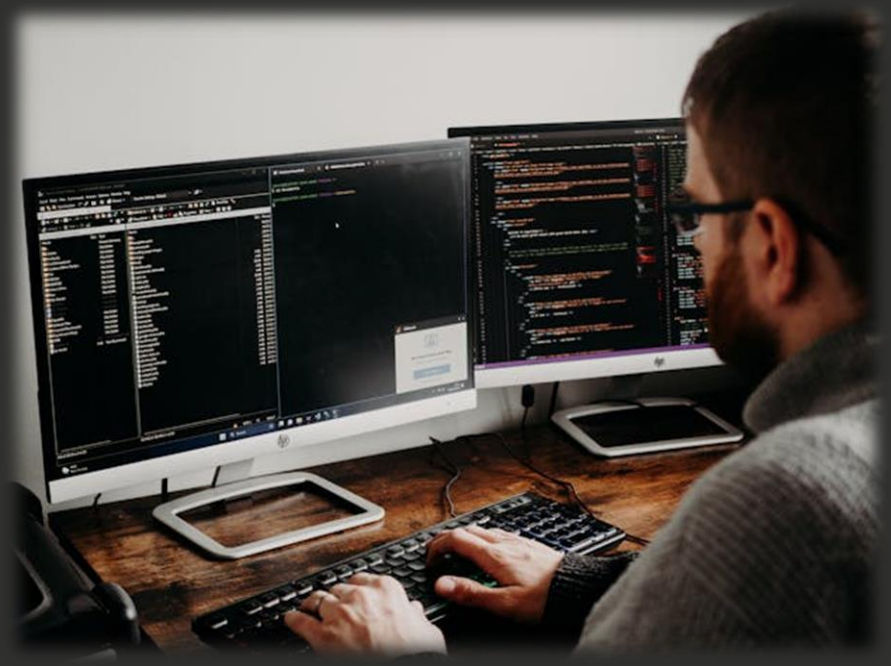
Introdução ao Git e Github (versionamento de código)

O que é o Versionamento de Código?

- **Versionamento de código** é o processo de **registrar** e **controlar** todas as **alterações** feitas em um código-fonte ao longo do tempo.

- Além disso, ele permite acompanhar o **histórico** de mudanças, voltar a **versões anteriores**, identificar quem fez cada alteração e trabalhar em equipe **sem sobrescrever** o trabalho de outras pessoas.

- **Git** e **GitHub** são amplamente utilizados no desenvolvimento de software para gerenciamento de código, colaboração eficiente e controle de versões, tornando o processo de desenvolvimento mais organizado e ágil.



O que é Git?

- **Git** é um **sistema de controle de versão** distribuído que permite rastrear mudanças no código-fonte durante o desenvolvimento de software.

- Criado por **Linus Torvalds** em 2005, o sistema Git facilita a colaboração entre desenvolvedores, o gerenciamento de versões e a recuperação de histórico de alterações.

O que é o Github?

- **GitHub** é uma plataforma de hospedagem de código baseada na web que usa Git para controle de versão. Criado em 2008, oferece uma série de recursos para colaboração e gerenciamento de projetos.

- Ele permite que desenvolvedores trabalhem de forma colaborativa, registrando alterações no código, acompanhando o histórico de versões e resolvendo conflitos de maneira organizada.

Exemplo de perfil no Github

Laura Grassi
lauragrassig

Software Developer at X-Team | Content Creator | 27 years old

Edit profile

2.2k followers · 12 following

@x-team
São Paulo - Brasil
16:53 (UTC -03:00)
https://codepen.io/lauragrassig
@kibumLaura

Achievements

lauragrassig / README.md

👋 Hi there

I'm Laura Grassi, a passionate software developer and tech enthusiast. I currently work as a Senior Software Developer at XTEAM. With over 8 years of experience in the field, I've had the opportunity to work on various exciting projects and explore different technologies.

🚀 Expertise

Passionate about crafting interactive and intuitive user experiences. I specialize in front-end web development, combining modern frameworks with best practice design patterns. With almost 7 years dedicated to honing my skills in front-end technologies, I've led teams, designed system guidelines, and upheld the highest standards in web performance and SEO.

In my professional journey, I've proudly served as a Front-end Technical Lead as well, a role where my technical expertise met the challenges of team management. Beyond crafting top-tier user interfaces, I embraced the responsibilities of leadership, guiding my team with strategic oversight, and ensuring projects were executed efficiently.

In addition to my professional pursuits, I've embraced the role of a "content creator" within the technology sphere. My approach is distinct; I aim to offer a fresh perspective on tech, blending insights with a dash of humor.

Outside of the professional environment, I nurture a unique passion: creating "art" using exclusively CSS. This activity not only sharpens my technical prowess but also allows me to blend programming and creativity in a uniquely captivating way.

💻 Tech Stack

JAVASCRIPT | TYPESCRIPT | HTML5 | CSS3 | REACT | VUEJS | NEXT | NUXT
SASS | LESS | MUI | STYLED-COMPONENTS | VUETIFY | THREEJS | JIRA | NOTION
TRELLO | POSTMAN | BABEL

🌐 Socials:

Instagram | LinkedIn | TikTok | Twitter

> "Do, or do not. There is no "try" — Yoda

Principais Características do Git

- **Controle de Versão Distribuído:** Cada desenvolvedor possui uma cópia completa do repositório, incluindo seu histórico, o que possibilita trabalhar offline e colaborar de forma descentralizada.

- **Branching e Merging:** Permite criar **ramificações** (**branches**) para desenvolver funcionalidades separadas e depois **mesclá-las** (**merge**) ao branch principal.

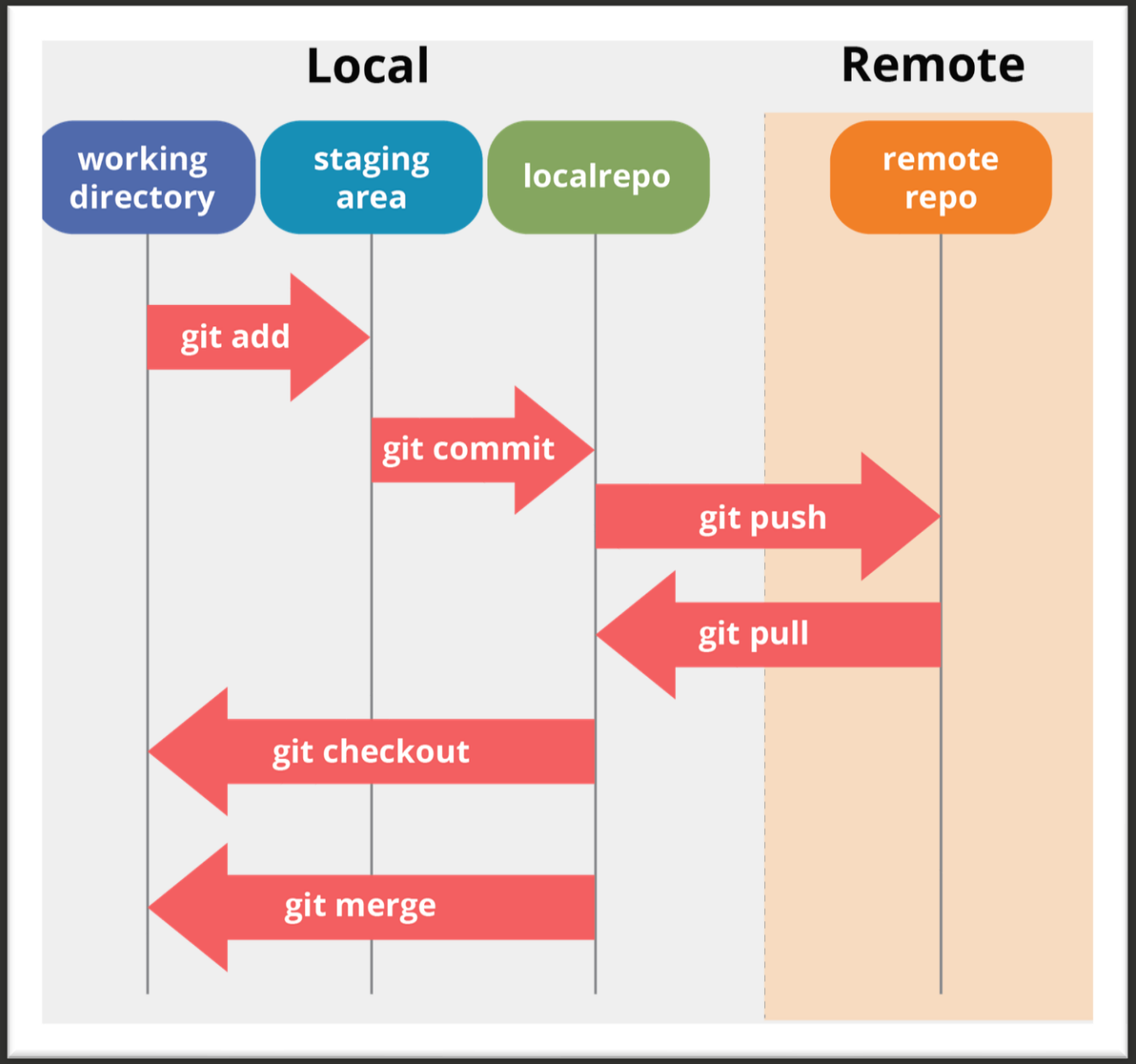
- **Registro de Histórico:** Armazena um histórico detalhado de todas as alterações feitas no código, facilitando a revisão e a recuperação de versões anteriores.

Principais Características do Github

- **Repositórios Remotos:** Hospeda repositórios Git na nuvem, permitindo o acesso e a colaboração em projetos de qualquer lugar.

- **Interface Web:** Oferece uma interface gráfica para gerenciar repositórios, visualizar o histórico de commits, revisar pull requests e mais.

- **Colaboração:** Facilita a colaboração e discussões, permitindo que os desenvolvedores revisem e discutam alterações antes de integrá-las.


Local
Remote
working directory
staging area
localrepo
remote repo
git add
git commit
git push
git pull
git checkout
git merge

Commit

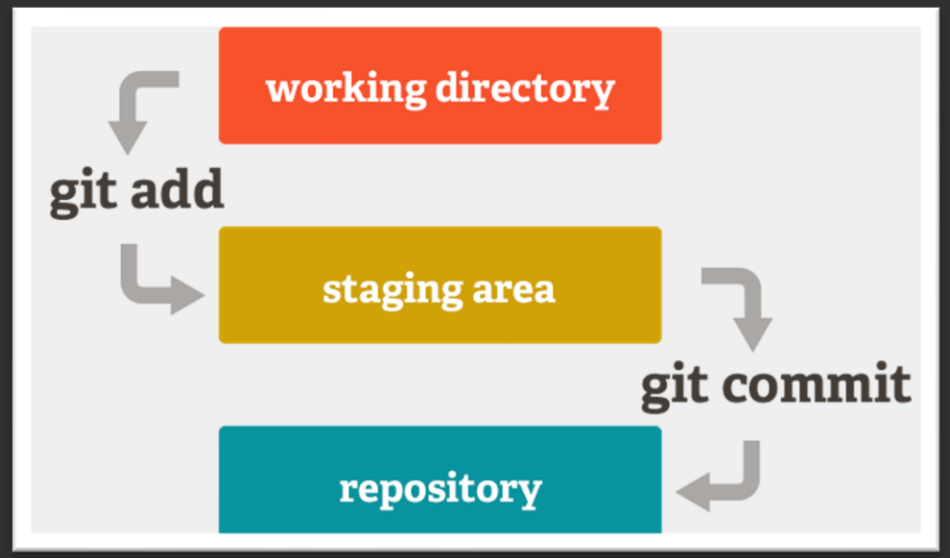
- Commit é o ato de **registrar** oficialmente um conjunto de alterações no repositório Git local.

- Cada commit cria um marco no histórico do projeto, contendo as mudanças realizadas, uma mensagem descritiva e informações de autoria, permitindo rastrear e restaurar versões anteriores do código.

Stagging area (índice de preparação)

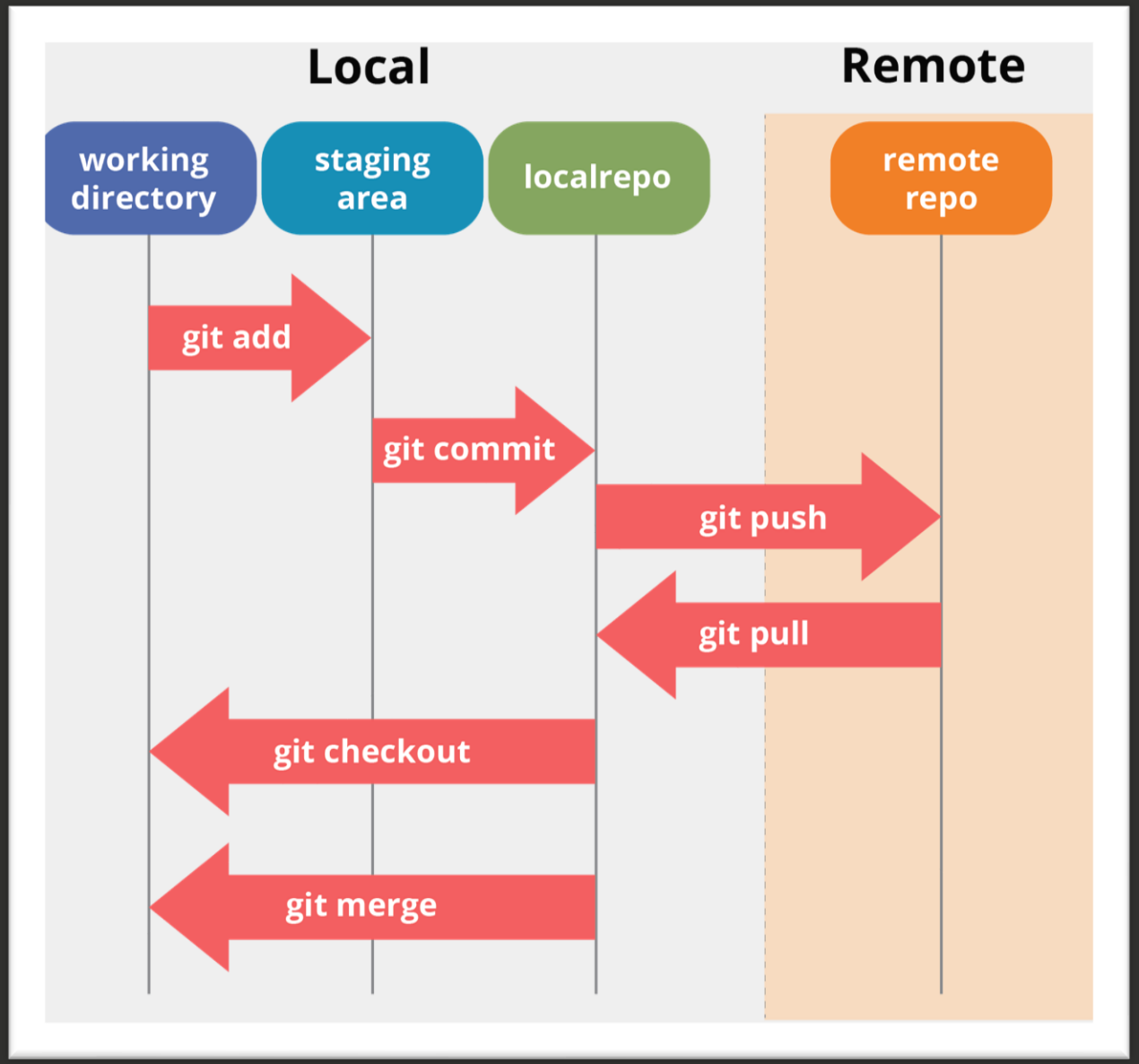
- O índice de preparação ou *staggind area* é uma área intermediária entre o diretório de trabalho (working directory) e o seu repositório Git.

- Funciona como uma área de seleção onde o desenvolvedor escolhe quais alterações do diretório de trabalho farão parte do próximo **commit**.



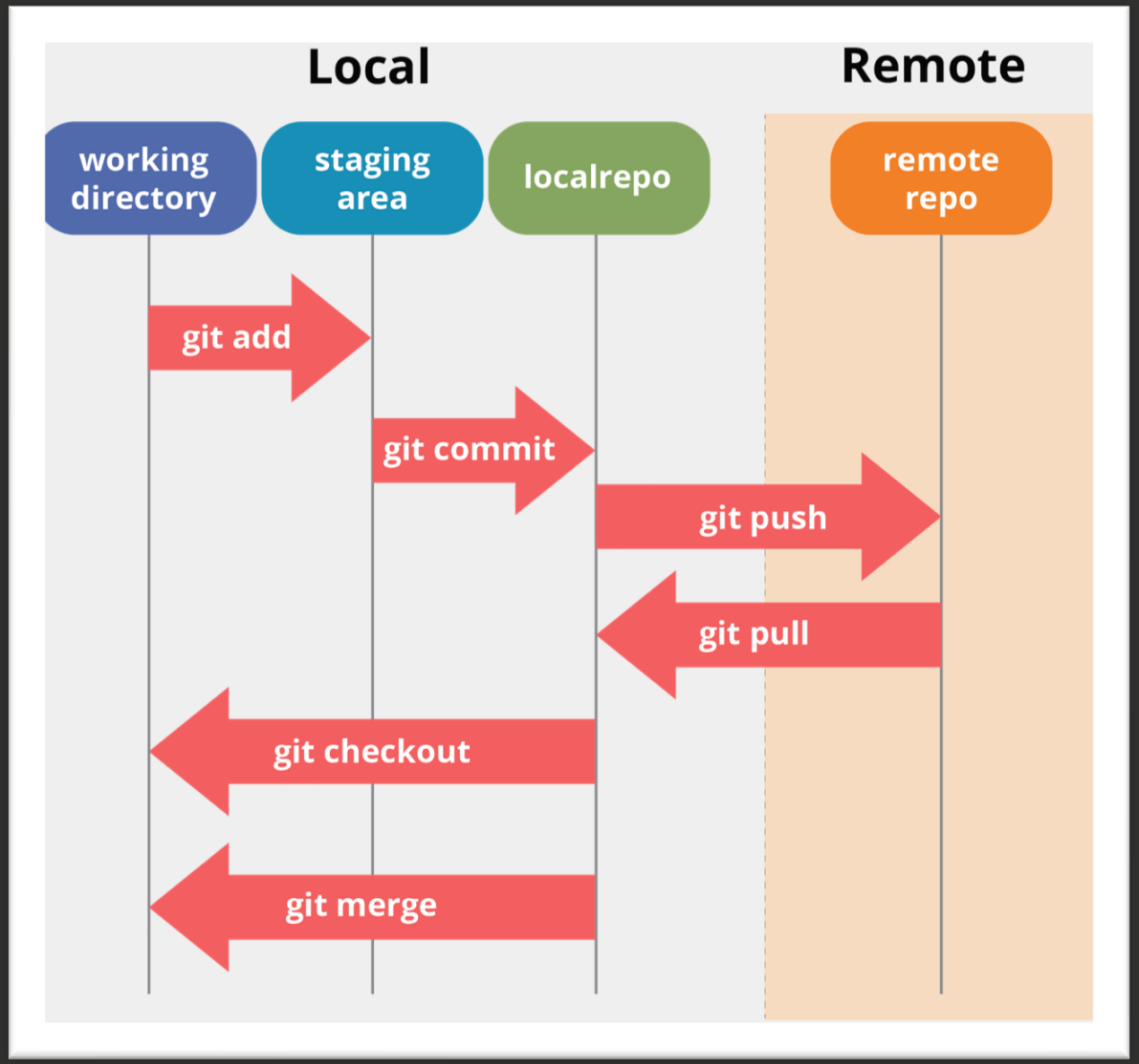
Push

- **Push** é a ação de **enviar os commits** do repositório local para um repositório remoto, como o GitHub. Ele torna as alterações visíveis para outros desenvolvedores, possibilitando a colaboração, o backup do código e a sincronização do projeto entre diferentes máquinas.



Pull

- **Pull** é a ação de buscar e integrar alterações que estão em um repositório remoto para o repositório local. Ao executar um pull, o Git **baixa as versões mais recentes do código** e, em seguida, mescla (merge) essas alterações com a branch local correspondente.



Branch (ramificação)

- Uma branch é uma **linha de desenvolvimento paralela** dentro de um repositório Git. Ela permite criar, testar e modificar funcionalidades sem afetar o código principal, geralmente localizado na branch **main** (ou **master**).

- As branches facilitam o trabalho em equipe, a organização de novas funcionalidades e a correção de erros, pois, após testadas, as alterações podem ser integradas ao projeto principal por meio de um **merge**, mantendo o histórico do código organizado e seguro.

Links para acesso / download

- **Git:**
  https://git-scm.com/install/windows

- **Github:**
  https://github.com/?locale=pt-BR

- **Github desktop:**
  https://desktop.github.com/download/

- **Documentação Github:**
  https://docs.github.com/pt/get-started/start-your-journey/hello-world

Aula 03.2:

Principais comandos do Git

Principais comandos do Git

Definir as configurações do usuário:

```
git config --local user.name Diego
```

```
git config --local user.email diego@email.com.br
```

Baixar um repositório criado no Github:

```
git clone --branch <branch_name> <repository_url>
```

```
git clone --branch 1.0 [url]
```

Exemplo simplificado:

`git clone https://github.com/maxxdiego/01_html5`

Principais comandos do Git

Verificar a **branch** que você está:

```
git branch
```

Baixar as atualizações do repositório:

```
git pull
```

Adicionar **um** arquivo criado para a **staging area**:

```
git add <file>
```

```
git add README.md
```

Adicionar **todos** os arquivos criados para **staging area** :

```
git add .
```

Principais comandos do Git

Verificar o status dos arquivos no repositório:

```
git status
```

Adicionar um **commit** ao repositório:

```
git commit -m "Criar o arquivo readme"
```

O comando -m permite que insira a mensagem de commit diretamente na linha de comando.

Enviar os commits locais, para um repositório remoto:

```
git push <remote> <branch>
```

```
git push origin 1.0
```

Exemplo simplificado:
`git push` *(somente)*

Principais comandos do Git

Criar nova **branch**:

```
git checkout -b <branch>
```

```
git checkout -b desenvolvimento
```

Mudar de **branch:**

```
git switch <branch>
```

```
git switch 1.0
```

Principais comandos do Git

Mesclar o histórico de commits de uma branch criada para a branch **main** (principal):

Primeiro mude para a branch main com o comando:

git switch main *(ou master)*

Após isso:

```
git merge <branch_name>
```

```
git merge desenvolvimento
```

Nesse caso, a branch "***desenvolvimento***" será mesclada a branch principal "***main***".

Principais comandos do Git

Retirar um arquivo do **staging area**:

```
git restore --staged <file_path>
```

```
git restore --staged index.html
```

Nesse caso, o arquivo "***index.html***" será retirado do **índice de preparação (stagging área)** e não será adicionado ao novo commit.

Principais comandos do Git

Para o dia a dia, durante o desenvolvimento. grave a seguinte sequência de comandos:

git add .

git commit –m "*Comentário*..."

git push

Ao baixar o repositório em uma máquina diferente, para sincronizar as modificações:

git pull

Exercício de fixação - Aula 03:

https://forms.office.com/r/83Ex4MrJqj

Desenvolvimento Web 1

Prof. Diego Max